```
      case LeaveHotel =>
        context.system.receptionist ! Receptionist
          .Deregister(HotelDesk.goldenKey, context.self)    ❶
        Behaviors.same
}}}
```

❶ Actor requesting deregistration

When an actor stops, it is automatically deregistered. So if the actor no longer plays a role in your application after leaving the hotel, stopping will do. You can do the following:

```
object VIPGuest {

  ...

  def apply() = Behaviors.receiveMessage[Command] {
      case EnterHotel => ...
      case LeaveHotel => Behaviors.stopped
}}}
```

DEFINITION *Routers* are proxies that forward messages to actors.

Routers are essential to scale up or out: for example, if you want to scale up a task and create multiple instances of the same task. Routers help you do this by forwarding messages to these tasks and splitting the work. At the beginning of the next section, you learn three reasons to use routers to control message flow:

- Performance
- Message content
- State

Then you learn how to create routing processes for each of these patterns. If you need a routing solution for performance or scaling reasons, you should use Akka's built-in routers, because they are optimized.